

Minimum Common String Partition Revisited

Haitao Jiang^{1,2}, Binhai Zhu¹, Daming Zhu², and Hong Zhu³

¹ Department of Computer Science, Montana State University,
Bozeman, MT 59717-3880, USA
{htjiang,bhz}@cs.montana.edu

² School of Computer Science and Technology, Shandong University, Jinan, China
dmzhu@sdu.edu.cn

³ College of Software, East China Normal University, Shanghai, China
hzhu@sei.ecnu.edu.cn

Abstract. Minimum Common String Partition (MCSP) has drawn much attention due to its application in genome rearrangement. In this paper, we investigate three variants of MCSP: $MCSP^c$, which requires that there are at most c symbols in the alphabet; d -MCSP, which requires the occurrence of each symbol to be bounded by d ; and x -balance MCSP, which requires the length of blocks not being x away from the average length. We show that $MCSP^c$ is NP-hard when $c \geq 2$. As for d -MCSP, we present an FPT algorithm which runs in $O^*((d!)^k)$ time. As it is still unknown whether an FPT algorithm only parameterized on k exists for the general case of MCSP, we also devise an FPT algorithm for the special case x -balance MCSP parameterized on both k and x .

1 Introduction

String comparison has drawn a lot of attention due to its applications in computational biology, text processing and compression. In this paper, we revisit the Minimum Common String Partition (MCSP) problem which has a close relation to the genome rearrangement problems such as Edit distance, Sorting by Reversals and Transpositions, etc.

A partition P of a string X is a sequence $P = \langle P_1, P_2, \dots, P_m \rangle$ of strings whose concatenation is equal to X , that is $P_1 P_2 \dots P_m = X$. The strings P_i are called the blocks of P . Given a partition P of a string X and a partition Q of a string Y , we say that the pair $\pi = (P, Q)$ is a common partition of X and Y if Q is a permutation of P , that is, there exists a permutation σ on $[m]$ such that $P_i = Q_{\sigma_i}$, $1 \leq i \leq m$. The *minimum common string partition problem* is to compute a common partition of X, Y with the minimum number of blocks.

In the Minimum Common String Partition (MCSP) problem, we are given two strings X and Y of length n over an alphabet Σ . Let each symbol appear the same number of times in X and Y . Throughout this paper, we assume that X and Y satisfy this condition. Clearly, this is a sufficient and necessary condition for X and Y to have a common string partition. For example, two strings $X = beabcdb$ and $Y = abdbebc$ have a common partition $(\langle b, e, ab, c, db \rangle, \langle ab, db, e, b, c \rangle)$. There

are several variants of MCSP. The restricted version where each letter occurs at most d times in each input string, is denoted by d -MCSP. Another important version where the input strings are over an alphabet with size bounded by c , is abbreviated as $MCSP^c$. We also introduce a feasible version where the resulting blocks in the partition is designated to have nearly even length, we call it x -balance MCSP. For the x -balance MCSP problem, if the optimum solution has k blocks, the length of each block ranges from $n/k - x$ to $n/k + x$. The signed minimum common string partition problem (SMCSP) is also a variant of MCSP in which each letter of the two input strings is given a “+” or “-” sign. For a string S with signs, let $-S$ denote the reverse of S , with each letter sign flipped. The common partition has a little bit of difference from the original where $P_i = Q_{\sigma_i}$ or $P_i = -Q_{\sigma_i}$.

Related work

The problem d -MCSP is well studied. 2-MCSP (and therefore MCSP) is NP-hard; moreover, APX-hard [8]. Several approximation algorithms are known for the problem [8,10]. Chen et al. [1] studied the problem of computing signed reversal distance with duplicates (SRDD). They introduced the signed minimum common partition problem as a tool for dealing with SRDD and observed that for any two related signed strings X and Y , the size of a minimum common partition and the minimum number of reversal operations needed to transform X and Y , are within a multiplicative factor 2 of each other. Kolman [12] devised an $O(d^2)$ -approximation algorithm running in $O(n)$ time for SRDD.

Chrobak et al. [3] analyzed the greedy algorithm for MCSP, they showed that for 2-MCSP, the approximation ratio is exactly 3, for 4-MCSP the approximation ratio is $\Omega(\log n)$; for the general MCSP, the approximation ratio is between $\Omega(n^{0.43})$ and $O(n^{0.67})$. The same bounds apply for SMCSP. Kaplan and Shafrir [9] improved the lower bound to $\Omega(n^{0.46})$ when the input strings are over an alphabet of size $O(\log n)$. Kolman [11] described a simple modification of the greedy algorithm; the approximation ratio of the modified algorithm is $O(p^2)$ for p -MCSP. Christie and Irving [2] proved that the problem of computing (unsigned) reversal distance is NP-hard for binary strings, it turns out that this problem has some connection to $MCSP^c$.

On the framework of parameterized complexity, Damaschke first solved MCSP by an FPT algorithm with respect to parameters k (size of the optimum solution), r (the repetition number) and t (the distance ratio depending on the shortest block in the optimum solution) [4].

Our Contribution

We prove that the problem $MCSP^c$ is NP-complete when $c \geq 2$. For d -MCSP, we introduce an equivalence condition for the common partition and present an FPT algorithm for that problem with running time $O^*((d!)^k)$. When the MCSP solution is supposed to be x -balance, we devise an FPT algorithm with parameter k and x which runs in $O((2x)^k k!n)$ time.

2 Preliminaries

As aforementioned, we recall the formal definition of MCSP.

Minimum Common String Partition

Input: two strings X, Y over an alphabet Σ

Question: Can X have a partition P and Y have a partition Q such that Q is a permutation of P ?

We say that the elements from Σ *occur* or *appear* in X and Y . A specific occurrence of some element is called a *symbol*. The strings in the partition are called blocks. There is a *break* or *cut* between two consecutive blocks in X or Y respectively.

A *duo* is a substring of length two. A *specific duo* is an occurrence of a duo in X or Y . Two specific duos are continuous if they form a substring of length three. A match is a pair $(a_i a_{i+1}, b_j b_{j+1})$ of specific duos, one from X and the other one from Y , such that $a_i = b_j$ and $a_{i+1} = b_{j+1}$. We also say that the pair of specific duos are matched if they form a match. Two matches form a *conflict* if they cannot be realized at the same time.

An FPT (Fixed-Parameter Tractable) algorithm for an optimization problem Π with optimal solution value k is an algorithm which solves the problem in $O(f(k)n^c)$ time, where f is any function only on k , n is the input size and c is some fixed constant not related to k . For convenience we also say that Π is in FPT. More details on FPT algorithms can be found in [5,6].

It is open whether an FPT algorithm exists for the general $MCSP$ with k being the unique parameter, so we try to consider variants of it by the use of additional parameters.

This paper is organized as follows. In Section 2, we present the NP-completeness of $MCSP^c$. In Section 3, we present FPT algorithms for the two special cases. In Section 4, we conclude the paper with several open questions.

3 Hardness for $MCSP^c$

In this section, we prove that $MCSP^c$ is NP-complete when $c \geq 2$ by a reduction from 3-PARTITION [7]. Firstly, we go over the formal definition of 3-PARTITION.

3-PARTITION

Input: Positive integers n and B , and positive integers set $A = \{a_1, a_2, \dots, a_{3n}\}$, with $B/4 < a_i < B/2$ and $\sum_{a_i \in A} a_i = nB$.

Question: Can A be partitioned into n disjoint sets $S_1, S_2, \dots, S_n$ such that, for $1 \leq i \leq n$, $\sum_{a_j \in S_i} a_j = B$.

The problem 3-PARTITION is strongly NP-hard: that is, there is a polynomial $p(n)$ such that it is still NP-hard when all the a_i 's are at most $p(n)$. Our reduction is polynomially bounded for instances of this type.

Given an instance of 3-PARTITION with integers (weights) $a_1, a_2, \dots, a_{3n}$, we construct two strings X, Y for an instance of $MCSP^2$ as follows

$$X = 11110^{a_1}11110^{a_2} \dots 11110^{a_{3n}}$$

$$Y = (0^B 1)^n 1^{11n}$$

For the sake of clarity, we first make the following claims.

Claim. Any block in the common string partition has the form $0^p 1$, 10^p , 0^p or 1^q where $p, q \geq 1$.

Proof. Suppose to the contrary that there is a common block H with the form $0^p 1^q 0^r \dots$, then from the format of X and Y , q is either equal to one or equal to four. But H cannot be a substring of X when $q = 1$ and cannot be a substring of Y when $q = 4$. Similarly, any block in the common partition will not have the form $1^q 0^p 1^s \dots$. Consequently, all the common blocks must be of the form $0^p 1$, 10^p , 0^p and 1^q , as there is only one 1 between two 0's in Y . The claim holds. $\square$

Claim. The interior '11' from '1111' in X is matched to '11' from 1^{11n+1} in Y .

Proof. From the first claim we conclude that, if a common block contains 0, then it contains at most one 1. So the interior two 1's from '1111' must be in the common blocks containing only 1's. Since there are no consecutive 1's in Y besides the substring 1^{11n+1} , the claim is certainly true. $\square$

Theorem 1. *MCSP² is NP-complete.*

Proof. We prove that the 3-PARTITION has an precise partition if and only if X, Y have a common partition with $6n$ blocks.

On the necessary side, assume that $S_1, S_2, \dots, S_n$ satisfy $\sum_{a_j \in S_i} a_j = B$, for all i . For any $S_i = \{a_p, a_q, a_r\}$, we obtain three blocks from X : $0^{a_p}, 0^{a_q}, 0^{a_r} 1$ and one block from Y : $0^B 1$. Obviously, the block from Y can be divided into three blocks corresponding to the blocks from X . Then there remains $2n$ blocks of the form '1111' and n blocks of the form '111' in X and all these blocks are separated (not adjacent). The unique block left in Y is 1^{11n+1} . Consequently, we obtain a common partition of X, Y with $6n$ blocks.

On the sufficient side, from the two claims, we can see that all the 0^{a_i} and the interior substring '11' of '1111' are in different blocks in the common partition. So there are at least $6n$ blocks in any common partition. If the number of blocks is exactly $6n$, then every block of the form 0^B in Y is matched to exactly three blocks of the form 0^{a_i} , which means it is a yes 3-PARTITION instance. $\square$

Since we can construct an instance of $MCSP^c$ when $c > 2$ from an instance of $MCSP^2$ by adding the same substring composed of symbols other than those in $MCSP^2$ at the ends of the two input strings, we can easily obtain the following corollary.

Corollary 1. *MCSP^c is NP-complete for $c \geq 2$.*

4 The FPT Algorithms

In this section, we design FPT algorithms for two variants of MCSP. As it is still unknown whether MCSP has an FPT algorithm parameterized only on k , we hope these algorithms could help shed light on answering this open question. On the other hand, these two variants are closely related to genome rearrangement problems, hence are meaningful practically.

4.1 On d -MCSP

In this subsection, we describe the FPT algorithm for d -MCSP.

First, let us recall the definition of duos and matches. Note that two matches with the continuous specific duos in one string are not in conflict if and only if these duos are matched to two continuous specific duos in the other string.

Lemma 1. *Each duo appears the same number of times in X and Y in the common partition.*

Proof. Otherwise, there must be such a specific duo that is unmatched in the partition. $\square$

Lemma 2. *If each duo appears the same number of times in a partition of X and a partition of Y , then two matches are conflict if and only if the specific duos are continuous in one string and not continuous in the other.*

Proof. A conflict match means that there exists a symbol which is matched to two occurrences of that symbol in the matches. Clearly in this case the specific duos cannot be continuous in both strings. $\square$

Lemma 3. *Given a pair of partition $\pi = (P, Q)$ for X and Y , if all specific duos are matched and all matches are not conflict then π is a common partition.*

Proof. For any block $G = g_1g_2 \cdots g_m$ in P , since all the specific duos $g_i g_{i+1}$ are matched, there must be matches $(g_i g_{i+1}, h_j h_{j+1})$ and $(g_{i+1} g_{i+2}, f_k f_{k+1})$ that can be realized at the same time. Since $g_i g_{i+1}$ and $g_{i+1} g_{i+2}$ are continuous duos, $h_j h_{j+1}$ and $f_k f_{k+1}$ should be continuous too; that is, h_{j+1} and f_k are the same symbol. As the above analysis holds for all i , we can see that there is a block $H = h_1 h_2 \cdots h_m$ in Q such that $G = H$. The lemma holds. $\square$

Our FPT algorithm just follows from the three lemmas. The rough idea is to cut redundant specific duos such that all duos appear the same number of times in X and Y , then cut one of the continuous duos that corresponds to two conflict matches. In the algorithm, we use arrays C and D indexed by duos to store the number of occurrence of each duo in X and Y respectively. That is, $C_{ab} = r$ means ab appears r times in X .

Theorem 2. *Algorithm Cut-Duos runs in $O^*((d!)^k)$ time.*

Proof. From the algorithm we branch on step 2, 3 and 5. As for the cost of the algorithm, Step 2 has recurrence relation

$$f(k) = \binom{r}{s} (s!) f(k - (r - s)).$$

Step 3 has a similar recurrence relation. Step 5 has recurrence relation

$$f(k) = 2f(k - 1).$$

Since $r, s \leq d$ and $f(k)$ achieves its maximum value when $r - s = 1$, so $f(k) = O^*((d!)^k)$. $\square$

Algorithm. Cut-Duos

Input: two strings X, Y such that each symbol appears at most d time in each. string

Output: A common partition (P, Q)

- 1 Compute the number of occurrence of each duo in X and Y , and store them in arrays C and D .
- 2 For every duo ab in X , if $r = C_{ab} > D_{ab} = s$
 - 2.1 Choose $r - s$ ab 's in X , cut them.
 - 2.2 Compute matches between the remaining ab 's in X and Y .
- 3 For every duo ab in Y , if $r = C_{ab} < D_{ab} = s$
 - 3.1 Choose $s - r$ ab 's in Y , cut them.
 - 3.2 Compute matches between the remaining ab 's in X and Y .
- 4 For two conflict matches (ab, ab) and (bc, bc) with the common symbol b in X and Y .
- 5 Choose one duo from ab and bc , cut it in both X and Y .
- 6 Return the remaining blocks in X and Y respectively as the common partition (P, Q) .

4.2 On x -Balance MCSP

In this subsection, we deal with the x -balance MCSP problem. For the specific applications in genome arrangement, due to the relation between MCSP and genome rearrangement, each block corresponds to a gene and each gene should have roughly the same gene content, so the blocks are supposed to have nearly even lengths, possibly with some small deviation. Assume that the optimal solution size is k , then we can see that every block is of some length between $(n/k) + x$ and $(n/k) - x$.

The rough idea of the FPT algorithm is to cut the input string into blocks of length ranging from $(n/k) - x$ to $(n/k) + x$, then compute all possible matches between the resulting k blocks. In the algorithm, assume that the length of the i th block is $L(i)$. When we refer to *positions* from integer u to v ($u < v$) in X , we mean the set of duos $\{X_u X_{u+1}, X_{u+1} X_{u+2}, \dots, X_{v-1} X_v\}$.

Theorem 3. *Algorithm Cut-Depending-on-Length runs in $O((2x)^k k!n)$ time.*

Proof. From the algorithm we branch on step 2, 3 and 4. Step 2 runs $(2x)^{k-1}$ times. Step 3 has a similar running time. Step 4 has $k!$ possibilities. Step 5 runs in $O(n)$ time. So the total running time of the algorithm is $O((2x)^k k!n)$. $\square$

We comment that Algorithm Cut-Depending-on-Length works for other varieties of MCSP whenever they are x -balance.

5 Concluding Remarks

In this paper, we deal with some variants of MCSP. The first one, $MCSP^c$, is proved to be NP-complete when $c \geq 2$. For d -MCSP and x -balance MCSP, we devise FPT algorithms for them. The original MCSP problem seems difficult to solve with an FPT algorithm only on the parameter k , so some extra practical parameters which make the problem fixed-parameter tractable are meaningful. As d -MCSP has close relation to some specific genome rearrangement problem, we are currently seeking good approximations for the d -MCSP problem.

Algorithm. *Cut-Depending-on-Length*

Input: *an x -balance instance of MCSP*

Output: *A common partition (P, Q)*

- 1 Compute the scope of the length of blocks.
- 2 For input string X , for $1 \leq i \leq k - 1$
 - 2.1 Let S_0 be set of position from $n - (n/k + x)(k - i)$ to $n - (n/k - x)(k - i)$
 - 2.2 Let S_1 be set of position from $\sum_{0 \leq j < i} L(j) + (n/k - x)$ to $\sum_{0 \leq j < i} L(j) + (n/k + x)$
 - 2.3 Choose a duo from $S_0 \cap S_1$ and cut it.
- 3 For input string Y , for $1 \leq i \leq k - 1$
 - 3.1 Let S_0 be set of position from $n - (n/k + x)(k - i)$ to $n - (n/k - x)(k - i)$
 - 3.2 Let S_2 be set of position from $\sum_{0 \leq j < i} L(j) + (n/k - x)$ to $\sum_{0 \leq j < i} L(j) + (n/k + x)$
 - 3.3 Choose a duo from $S_0 \cap S_2$ and cut it.
- 4 Compute a one-to-one mapping between the resulting blocks in X and Y .
- 5 Check whether all the two blocks in the map are common.

Acknowledgments

This research is partially supported by NSF grant DMS-0918034, and by NSF of China under grant 60928006. We also thank anonymous reviewers for some constructive comments.

References

1. Chen, X., Zheng, J., Fu, Z., Nan, P., Zhong, Y., Lonardi, S., Jiang, T.: Computing the assignment of orthologous genes via genome rearrangement. In: Proc. of the 3rd Asia-Pacific Bioinformatics Conf. (APBC 2005), pp. 363–378 (2005)
2. Christie, D.A., Irving, R.W.: Sorting strings by reversals and by transpositions. SIAM Journal on Discrete Mathematics 14(2), 193–206 (2001)
3. Chrobak, M., Kolman, P., Sgall, J.: The greedy algorithm for the minimum common string partition problem. In: Jansen, K., Khanna, S., Rolim, J.D.P., Ron, D. (eds.) RANDOM 2004 and APPROX 2004. LNCS, vol. 3122, pp. 84–95. Springer, Heidelberg (2004)

4. Damaschke, P.: Minimum Common String Partition Parameterized. In: Crandall, K.A., Lagergren, J. (eds.) WABI 2008. LNCS (LNBI), vol. 5251, pp. 87–98. Springer, Heidelberg (2008)
5. Downey, R., Fellows, M.: Parameterized Complexity. Springer, Heidelberg (1999)
6. Flum, J., Grohe, M.: Parameterized Complexity Theory. Springer, Heidelberg (2006)
7. Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman, New York (1979)
8. Goldstein, A., Kolman, P., Zheng, J.: Minimum common string partitioning problem: Hardness and approximations. In: Fleischer, R., Trippen, G. (eds.) ISAAC 2004. LNCS, vol. 3341, pp. 473–484. Springer, Heidelberg (2004); also in: The Electronic Journal of Combinatorics 12 (2005), paper R50
9. Kaplan, H., Shafrir, N.: The greedy algorithm for edit distance with moves. Inf. Process. Lett. 97(1), 23–27 (2006)
10. Kolman, P., Walen, T.: Reversal Distance for Strings with Duplicates: Linear Time Approximation Using Hitting Set. In: Erlebach, T., Kaklamanis, C. (eds.) WAOA 2006. LNCS, vol. 4368, pp. 279–289. Springer, Heidelberg (2007)
11. Kolman, P.: Approximating reversal distance for strings with bounded number of duplicates. In: Jedrzejowicz, J., Szepietowski, A. (eds.) MFCS 2005. LNCS, vol. 3618, pp. 580–590. Springer, Heidelberg (2005)
12. Kolman, P., Walen, T.: Approximating reversal distance for strings with bounded number of duplicates. Discrete Applied Mathematics 155(3), 327–336 (2007)